

Project Title: Ski Resort Gear Rental System
Course: CS340
Group: 80
Team Members: Joshua Cabalka, Jack Boland, Philip Gadsden
Project Step 5 Draft Version
Date: 03/9/2026

a) Project URL: <http://classwork.engr.oregonstate.edu:40405>

b) Feedback from Step 4 Draft:



Alexander Jones 4d

- Do all SELECT operations work as expected? In other words, is data from all the database tables displayed in the web application? If not, please describe which tables a data is not being displayed from.

Everything appears to work and tables are all populated.

- Is a RESET button/link implemented that properly resets the database? If not, describe the behavior/ what data is not being reset.

The RESET button worked for me after deleting and editing tables.

- For any additional CRUD operations that the team listed as operational, do they function as the team expects (e.g. if the team stated that a CRUD step worked, but you found an error, please tell them)?

All CRUD operations function as expected.

- Would a user easily be able to use the UI to complete the CRUD operations that the team listed as working? If not, please elaborate and provide helpful suggestions for how the UI can be improved.

I think that the UPDATE form is a little strange. If I wanted to edit an existing item or customer, I wouldn't want to have to fill out every detail from scratch. I would expect it to be filled with the information that is already in the table and then choose which sections I want to change. This way there is less room for error.

- What suggestions do you have for the team in any areas where they are blocked or having difficulty? Detailed, helpful feedback will receive higher credit. If the team is not blocked or is having difficulty, encouraging and supportive comments would be a better response than NO feedback.

I think it would make sense to have an Edit and Delete button next to each row of your tables instead of having to select from a drop down or go through a whole process just to edit one thing. This would also clean up the UI a lot.

All comments are my own work.

♡ Reply ...



Devin Gaughan 4d

Hi Joshua, Jack, and Philip,

Fantastic work on your application. Looks like you guys put in a lot of work!

Here is my review of your Ski Gear Rental project:

Do All SELECT operations work as expected?

Yes. The application successfully populates browse pages for all seven entities, including intersection tables by pulling data through the query loader.

Is a RESET button/link implemented that properly resets the database?

Yes. The RESET button on the index page triggers a stored procedure that drops and recreates all tables before reinserting the base sample data.

For any additional CRUD operations that the team listed as operational, do they function as the team expects?

All CRUD operations are functioning through stored procedure calls, through you might want to double check that your run_action function continues to handle result draining to prevent out of sync errors.

Would a user easily be able to use the UI to complete the CRUD operations?

Definitely, yes. The UI is very user friendly, featuring a shared nav header on every page and useful dropdown menus for selecting customers and gear items.

What suggestions do you have for the team in any areas where they are block or having difficulty?

I'm not too sure what I could add as a far as suggestions because of how smoothly your system is running, but perhaps you could adding more descriptive labels in your dropdown menus, like showing a brand and model of the gear if you want to account for the inventory/database being hefty enough to where staff might need help identifying items more quickly. Otherwise, super well done. Awesome job guys!

Originality: All answers are my work.

♡ Reply ...



Solomon Jgarkava 4d

Hello Group 80,

To preface, I will be writing this review in the order that is given in the assignment description.

Yes all SELECT operations work as expected.

Yes there is a RESET button and it restores the database back to the initial sample data.

All CRUD functionality is implemented and works without any problems (in my testing).

A user may have a hard time navigating this website as it is very much technical and not quite at the level it would be during its release. I would suggest using AS aliases and displaying names rather than IDs in RentalOrders, RentalOrderItems, ServiceTickets, and ServiceTicketItems.

You have done a very good job at implementing all of the functionality of the program. I would advise your next to be improving readability and styling. Currently the webpage seems to serve a purely utilitarian purpose with no styling or usability enhancements. You have set yourself up very well by implementing all of the functionality first leaving you plenty of time to mess around with styling and other aesthetic aspects of the website.

This review is my original work.

♡ Reply ...



Cory Hungate 7d

Aloha Group 80! Holy smokes, you've implemented a lot!

I gave the site a pretty solid test, and everything seems to be working great! I tested every button I could see to test, and everything worked as indicated. The UI, while basic, was clean and I did not have any trouble identifying how to perform the CRUD operations.

I took a look at your file structure as well. I'm really impressed how you imported all your SQL queries into a python dictionary and then used helper functions to make the database connection, execute the queries, and then close the connection. It's really elegant and I would have never thought to do something like that.

I don't have a lot to add unfortunately. The site looks great and the code looks great. My only recommendation would be some minor changes for UX. Having to completely retype all the parameters into your different update functions is a bit of a pain. If you could work up a way to auto-fill those entries when you select an entry ID would be really slick. I would also alias some of the ID numbers under the rental orders, service tickets, and service ticket items pages that would make them much more user friendly.

Overall, great work!

This review represents original work.

♡ Reply ...

c) Changes based on feedback from step 4 draft:

- i) No changes were made based on the step 4 feedback for this draft. UI design and usability suggestions were considered but not deemed necessary for this submission since all of the required functionality is working and the current interface supports the required workflows.

d) Feedback from Step 3 Final

- i) Grader feedback received: Step 3 PDF submission was missing the ERD.

e) Changes Based on Feedback from Step 3 Final

- i) Added ERD to the PDF.

f) Fixes Based on Feedback from Step 3 Draft

- i) Added a shared navigation header on all pages to make it easier for users to move between pages without having to return to the index.
- ii) DELETE operations properly implemented.

g) Design Changes from Step 3

- i) Added DB reset stored procedure in its own SQL file. The DML file calls it when the UI frontend requests it. There is a single reset button on the main index.html page that will reset the database to just the example data.
- ii) Converted all CRUD operations into stored procedures and stored them in a separate SQL file. Updated DML file to call procedures from the procedures file.
- iii) Fixed run_action function to safely handle stored procedure calls by draining results set to avoid errors.

iv) Fixed UI schema enum mismatches.

h) Fixes Based on Feedback from Step 2

Feedback based on the Step 2 submission was positive, with many reviewers noting that the system design, entities, relationships, ERD and Schema were clear and aligned with project requirements. No major changes were made to the Schema or SQL.

i) Fixes Based on Feedback from Step 1

Feedback on the Step 1 submission was mainly positive, with reviewers agreeing that the system design, entities, relationships, and ERD were clear and aligned with requirements. Only minor issues were identified and addressed.

Reviewers noted that the overview could better illustrate system scale. The overview was updated to more explicitly describe operational scope, including individual gear tracking, multi-item rental orders, historical checkout and return records, and independent maintenance workflows.

An error was identified in the database outline where `customer_id` in the `rental_orders` table incorrectly referenced the `employees` table. This was corrected to reference `customers(customer_id)`, aligning the outline with the ERD and business logic.

A minor naming inconsistency was identified in the `rental_order_items` table, where the foreign key name did not match the rental order primary key naming convention. Fixed foreign key reference so `rental_orders.customer_id` correctly references `customers(customer_id)`.

Clarified meaning of `employees.is_active` (employee is currently employed/authorized to create orders and service tickets).

No other changes were required, as reviewers confirmed that entity modeling, relationships, naming conventions, and the ERD were correct.

j) Project Overview and Database Outline

This project models an admin-facing ski resort gear rental system designed to manage an equipment inventory of approximately 300 individual gear items across various categories, along with an average of 80 rental transactions per week and 20 maintenance service tickets per week.

The system tracks individual gear items to allow staff to monitor availability, condition, rental history, and service history at the item level. Rental activity is recorded through rental orders created by employees for customers, with each rental order capable of containing multiple gear items. A typical rental order contains 2–4 gear items. Historical rental data is preserved using timestamps that record when items are checked out and returned.

Maintenance and repair work is tracked independently using service tickets, which document service procedures performed on one or more gear items. Each service ticket includes timestamps for work start and completion, allowing the system to maintain a complete maintenance history separate from rental activity. Together, these components support both daily operational needs and long-term inventory and service tracking for the ski resort.

DATABASE OUTLINE

1. **customers:** Records customers who rent gear from the ski resort.

Attributes

- `customer_id`: INT, AUTO_INCREMENT, PRIMARY KEY, NOT NULL
- `first_name`: VARCHAR(100), NOT NULL
- `last_name`: VARCHAR(100), NOT NULL
- `phone`: VARCHAR(20), NULL
- `email`: VARCHAR(255), NULL

Relationships

- One-to-many (1:M) relationship with `rental_orders`

2. **employees:** Records employees who create rental orders and service tickets.

Attributes

- `employee_id`: INT, AUTO_INCREMENT, PRIMARY KEY, NOT NULL
- `first_name`: VARCHAR(100), NOT NULL
- `last_name`: VARCHAR(100), NOT NULL
- `role`: VARCHAR(50), NOT NULL
- `is_active`: BOOLEAN, NOT NULL, DEFAULT TRUE
- Indicates whether the employee is currently employed and authorized to create rental orders and service tickets

Relationships

- One-to-many (1:M) relationship with `rental_orders`
- One-to-many (1:M) relationship with `service_tickets`

3. **gear_items:** Records each physical piece of ski or snowboard-related equipment.

Attributes

- `gear_item_id`: INT, AUTO_INCREMENT, PRIMARY KEY, NOT NULL
- `category`: ENUM('snowboard', 'boots', 'helmet', 'other'), NOT NULL
- `brand`: VARCHAR(100), NOT NULL
- `model`: VARCHAR(100), NOT NULL
- `serial_number`: VARCHAR(100), UNIQUE, NOT NULL
- `size`: VARCHAR(20), NOT NULL
- `condition_grade`: ENUM('new', 'good', 'fair', 'needs_repair'), NOT NULL
- `status`: ENUM('available', 'rented', 'in_service', 'retired'), NOT NULL

- `acquired_at`: DATETIME, NOT NULL

Relationships

- Many-to-many (M:M) relationship with `rental_orders`
- Many-to-many (M:M) relationship with `service_tickets`

4. **rental_orders**: Records each rental checkout transaction.

Attributes

- `rental_order_id`: INT, AUTO_INCREMENT, PRIMARY KEY, NOT NULL
- `customer_id`: INT, NOT NULL, FOREIGN KEY → `customers(customer_id)`
- `created_by_employee_id`: INT, NOT NULL, FOREIGN KEY → `employees(employee_id)`
- `created_at`: DATETIME, NOT NULL

Relationships

- Many-to-one (M:1) relationship with `customers`
- Many-to-one (M:1) relationship with `employees`
- Many-to-many (M:M) relationship with `gear_items` (via `rental_order_items`)

5. **service_tickets**: Records maintenance and repair work performed on gear items.

Attributes

- `service_ticket_id`: INT, AUTO_INCREMENT, PRIMARY KEY, NOT NULL
- `opened_by_employee_id`: INT, NOT NULL, FOREIGN KEY → `employees(employee_id)`
- `status`: ENUM('open', 'in_progress', 'completed', 'canceled'), NOT NULL
- `created_at`: DATETIME, NOT NULL

Relationships

- Many-to-one (M:1) relationship with `employees`
- Many-to-many (M:M) relationship with `gear_items` (via `service_ticket_items`)

INTERSECTION TABLES

6. **rental_order_items**: Implements the many-to-many relationship between `rental_orders` and `gear_items`.

Attributes

- `rental_order_id`: INT, NOT NULL, FOREIGN KEY → `rental_orders(rental_order_id)`
- `gear_item_id`: INT, NOT NULL, FOREIGN KEY → `gear_items(gear_item_id)`
- `checked_out_at`: DATETIME, NOT NULL
- `due_at`: DATETIME, NOT NULL
- `returned_at`: DATETIME, NULL

Primary Key

- (rental_order_id, gear_item_id)

Business Rule

- A gear item may only appear in one active rental at a time. This is enforced by checking that gear_items.status = 'available' prior to checkout and updating the status to 'rented' while returned_at is NULL.

7. **service_ticket_items**: Implements the many-to-many relationship between service_tickets and gear_items.

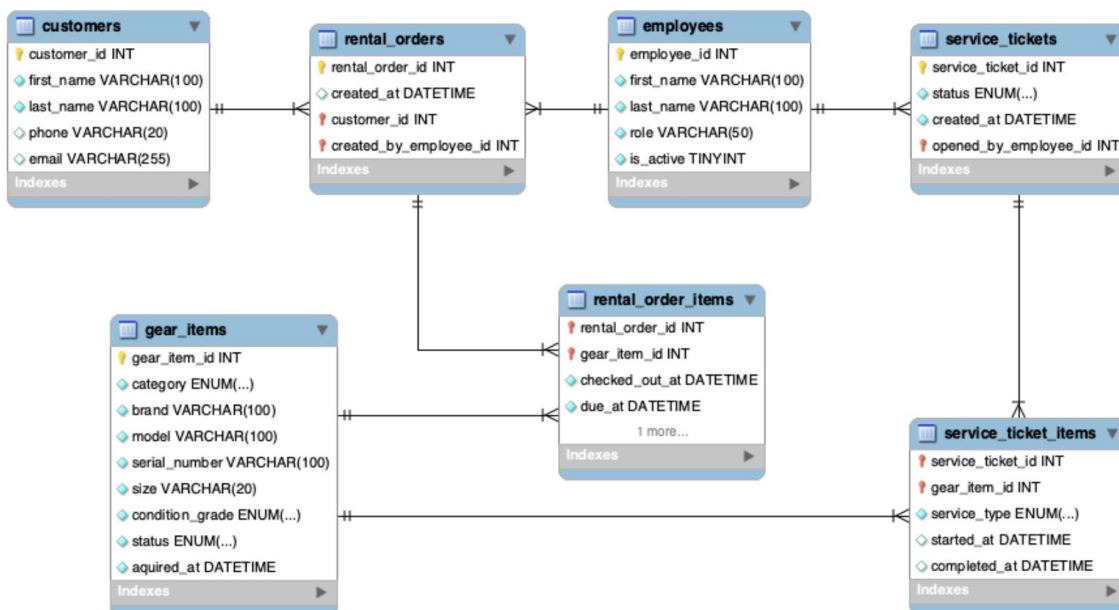
Attributes

- service_ticket_id: INT, NOT NULL, FOREIGN KEY → service_tickets(service_ticket_id)
- gear_item_id: INT, NOT NULL, FOREIGN KEY → gear_items(gear_item_id)
- service_type: ENUM('wax', 'tune', 'repair', 'binding_adjust', 'inspect'), NOT NULL
- started_at: DATETIME, NULL
- completed_at: DATETIME, NULL

Primary Key

- (service_ticket_id, gear_item_id)

k) Entity-Relationship Diagram (ERD)



I) Schema Diagram + Normalization (Write-up)

The following report was generated by joining rental orders, customers, employees, and gear items into a single flattened result set, where each row represents a gear item within a rental

	rental_order_id	order_created_at	customer_name	customer_phone	customer_email	employee_name	employee_role
1	1	2026-02-01 10:00:00	John Doe	541-555-0101	john.doe@example.com	Shaun White	rental_associate
2	1	2026-02-01 10:00:00	John Doe	541-555-0101	john.doe@example.com	Shaun White	rental_associate
3	1	2026-02-01 10:00:00	John Doe	541-555-0101	john.doe@example.com	Shaun White	rental_associate
4	2	2026-02-02 11:15:00	Noah Patel	<null>	noah.patel@example.com	Shaun White	rental_associate
5	3	2026-02-03 09:20:00	Mia Johnson	541-555-0103	<null>	Shaun White	rental_associate

order. This report reflects how the data might appear in a spreadsheet prior to normalization.

FIGURE 2: UNNORMALIZED RENTAL ACTIVITY SAMPLE REPORT DEMONSTRATING DATA REDUNDANCY AND UPDATE/INSERT/DELETE ANOMALIES PRIOR TO NORMALIZATION.

	serial_number	category	brand	model	size	checked_out_at	due_at	returned_at
1	SNB-0001	snowboard	Burton	Process	155	2026-02-01 10:05:00	2026-02-03 10:05:00	<null>
2	BTS-0101	boots	K2	Mysis	10	2026-02-01 10:05:00	2026-02-03 10:05:00	<null>
3	HLT-0201	helmet	Smith	Vantage	M	2026-02-01 10:05:00	2026-02-03 10:05:00	<null>
4	SNB-0001	snowboard	Burton	Process	155	2026-02-02 11:20:00	2026-02-04 11:20:00	2026-02-03 16:45:00
5	SNB-0001	snowboard	Burton	Process	155	2026-02-03 09:25:00	2026-02-05 09:25:00	<null>

In the sample report, order-level, customer, and employee attributes are repeated across multiple rows because each row represents a single gear item within a rental order. For example, rental order 1 appears three times, resulting in duplicated customer and employee data. This redundancy introduces update anomalies (e.g., a change to a customer's contact information must be applied to multiple rows), insert anomalies (new customers, gear items, or employees cannot be added independently without creating a full rental transaction), and delete anomalies (removing the final line item for an order may unintentionally remove associated order or customer information).

To convert the design to First Normal Form (1NF), repeating groups were eliminated by separating rental order data from individual gear items. Each rental order may contain multiple gear items, so a separate intersection table was introduced to represent one row per gear item per order. The composite key (rental_order_id, gear_item_id) uniquely identifies each row. With atomic attributes and no repeating groups, the design satisfies 1NF.

In the 1NF design, partial dependencies existed because order-level attributes depended only on rental_order_id, while gear attributes depended only on gear_item_id. To convert the design to Second Normal Form (2NF), these partial dependencies were removed by decomposing the schema into rental_orders, gear_items, and rental_order_items. Each non-key attribute now depends on the entire primary key of its table.

Transitive dependencies were identified where customer and employee attributes depended on non-key identifiers in rental orders (e.g., $\text{rental_order_id} \rightarrow \text{customer_id} \rightarrow \text{customer attributes}$). These dependencies were eliminated by moving customer information into a customers table and employee information into an employees table, referenced by foreign keys. After this decomposition, all non-key attributes depend only on the primary key of their respective tables, and the schema satisfies Third Normal Form (3NF).

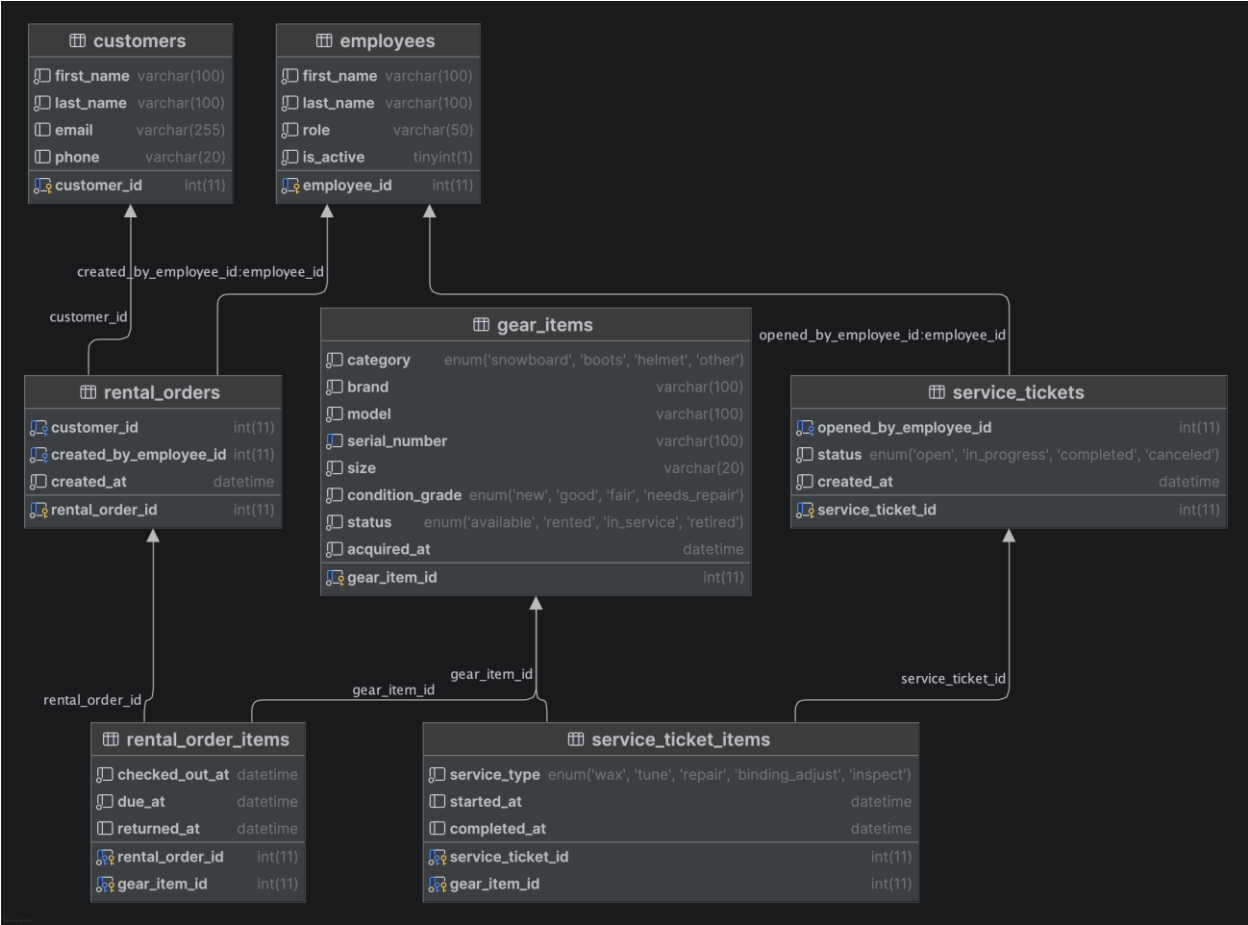


FIGURE 3: FINAL NORMALIZED DATABASE SCHEMA FOR THE SKI RESORT GEAR RENTAL SYSTEM.

m) Example Data

customers

	customer_id	first_name	last_name	email	phone
1	1	John	Doe	john.doe@example.com	541-555-0101
2	2	Noah	Patel	noah.patel@example.com	<null>
3	3	Mia	Johnson	<null>	541-555-0103

employees

	customer_id	first_name	last_name	email	phone
1	1	John	Doe	john.doe@example.com	541-555-0101
2	2	Noah	Patel	noah.patel@example.com	<null>
3	3	Mia	Johnson	<null>	541-555-0103

	employee_id	first_name	last_name	role	is_active
1	1	Shaun	White	rental_associate	1
2	2	Travis	Rice	service_tech	1
3	3	Lindsey	Vonn	manager	0

	gear_item_id	category	brand	model	serial_number	size	condition_grade	status	acquired_at
1	1	snowboard	Burton	Process	SNB-0001	155	good	available	2024-11-01 09:00:00
2	2	boots	K2	Mysis	BTS-0101	10	fair	rented	2023-12-10 10:30:00
3	3	helmet	Smith	Vantage	HLT-0201	M	new	in_service	2024-10-15 14:00:00
4	4	other	Dakine	Wrist Guard	OTH-0301	L	good	retired	2022-01-05 08:15:00

gear_items

	rental_order_id	customer_id	created_by_employee_id	created_at
1	1	1	1	2026-02-01 10:00:00
2	2	2	2	2026-02-02 11:15:00
3	3	3	3	2026-02-03 09:20:00

rental_orders
rental_order_items

	rental_order_id	gear_item_id	checked_out_at	due_at	returned_at
1	1	1	2026-02-01 10:05:00	2026-02-03 10:05:00	<null>
2	1	2	2026-02-01 10:05:00	2026-02-03 10:05:00	<null>
3	1	3	2026-02-01 10:05:00	2026-02-03 10:05:00	<null>
4	2	1	2026-02-02 11:20:00	2026-02-04 11:20:00	2026-02-03 16:45:00
5	3	1	2026-02-03 09:25:00	2026-02-05 09:25:00	<null>

service_ticket_items

	🔗 service_ticket_id ▾	÷ 🔗 gear_item_id ▾	÷ 📄 service_type ▾	÷ 📅 started_at ▾	÷ 📅 completed_at ▾
1		1	3 inspect	<null>	<null>
2		2	1 binding_adjust	2026-02-02 13:25:00	<null>
3		2	3 repair	2026-02-02 13:15:00	<null>
4		3	3 wax	2026-02-03 15:20:00	2026-02-03 16:05:00

service_tickets

	🔗 service_ticket_id ▾	÷ 🔗 opened_by_employee_id ▾	÷ 📄 status ▾	÷ 📅 created_at ▾
1		1	2 open	2026-02-01 08:30:00
2		2	2 in_progress	2026-02-02 13:00:00
3		3	2 completed	2026-02-03 15:10:00